Experiment – 1.2.1

Student Name: Damandeep Singh **UID: 24BDA7O313**

Branch: CSE (Data Science) **Section/Group:** 24 BDS-4(B)

Semester: 5th **Date of Performance:** 15-07-2026

Subject: Full Stack II **Subject Code:** 24CSP-337

1. Aim:

To design and implement a centralized state management system using Redux Toolkit for managing posts and platform-related data.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Framework/Libraries: React 18, Vite, Redux Toolkit, react-redux
- Browser: Google Chrome (for testing and rendering)

3. Objective:

This experiment introduces the concept of global state, state normalization, and structured state design to improve scalability, maintainability, and performance of frontend applications.

4. Theory:

Redux Toolkit's createSlice combines action creators and reducers in one place, and uses Immer internally so reducers can be written as direct state mutations while remaining immutable under the hood. Normalized state — storing entities as { byId, allIds } instead of a plain array — gives O(1) lookup and update by id, rather than O(n) array scans, which matters as the number of posts/drafts grows.

5. Procedure/Algorithm:

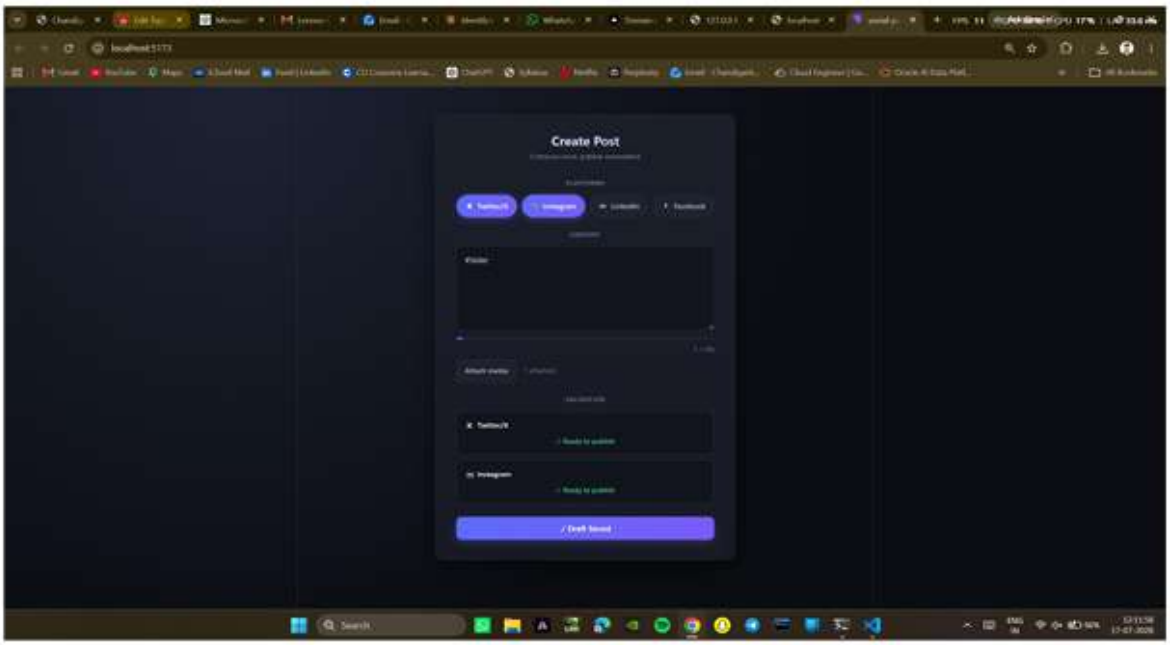
- Step 1: Define initialState with three normalized entities: posts, drafts, platforms.

- Step 2: Pre-populate platform metadata (Twitter/X, Instagram, LinkedIn, Facebook) with their constraints.
- Step 3: Write reducers: draftCreated (with a prepare callback to auto-generate id and timestamps), draftUpdated, draftDeleted, draftPublished, postRemoved.
- Step 4: Register the slice reducer inside configureStore to create a single global store.
- Step 5: Wrap the app in <Provider store={store}> so any component can access state via useSelector/useDispatch.

6. Code and Output:

```
const initialState = {
  posts:  { byId: {}, allIds: [] },
  drafts: { byId: {}, allIds: [] },
  platforms: {
    byId: {
      twitter: { id: 'twitter', name: 'Twitter/X', charLimit: 280, maxMedia: 4 },
      // ...instagram, linkedin, facebook
    },
    allIds: ['twitter', 'instagram', 'linkedin', 'facebook'],
  },
};

export const store = configureStore({
  reducer: { posts: postsReducer },
});
```



7. Conclusion:

A single, predictable source of truth was established for posts, drafts, and platform configuration. All state changes flow through typed actions, and any component in the tree can read or update this state without prop drilling — confirmed by dispatching draftCreated/draftUpdated from the UI and observing the Saved Drafts list update immediately.

8. Learning Outcomes:

- Learned to design normalized state shape (byId/allIds) for scalable entity management.
- Understood how Redux Toolkit's Immer integration simplifies reducer code.
- Learned to structure a slice covering multiple related entities in one store.